

fuClaw AIoT

技術參考手冊

Technical Reference for System Engineers & Developers

適用平台：Realtek AMB82-mini / HUB 8735 Ultra

Build：2026-06-07 作者：傅仲儀（ChungYi Fu） | 高雄，台灣

<https://github.com/fustyles/fuClaw>

第一章 系統架構總覽

1.1 執行環境

fuClaw 是運行在 Realtek AmebaPro2 (RTL8735B) SoC 上的嵌入式多模態 AI 代理人框架。硬體規格如下：

SoC	Realtek AmebaPro2 (RTL8735B)
RAM	128 MB DDR2 (SoC 內建)
Flash	16 MB SPI NOR (開發板外接)
作業系統	FreeRTOS (多工並行任務排程)
主要開發板	AMB82-mini / HUB 8735 Ultra

1.2 系統軟體堆疊

函式庫 / 元件	用途	備註
WiFi.h + WiFiSSLClient	WiFi 連線與 TLS/SSL 通訊	用於 Gemini API 與 MQTT (TLS)
PubSubClient	MQTT Broker 訊息收發	支援 QoS 0，非阻塞 TCP 模式
ArduinoJson	JSON 序列化與反序列化	tool_call 解析核心，8 KB 文件緩衝
FreeRTOS	多工任務排程	5 個並行 Task，各自獨立堆疊
VideoStream	相機畫格擷取	JPEG，320×240，透過 RTSP 串流
Base64	二進位→ASCII 編碼	圖片與語音嵌入 JSON 傳輸
AmebaFatFS	SD 卡檔案讀寫	持久化設定檔與對話記憶
AmebaServo	SG90 伺服馬達控制	PWM 角度控制，0–180°
DHT	DHT11 溫濕度感測	單次觸發讀值，需間隔 ≥1 s

1.3 FreeRTOS 任務架構

系統啟動後，setup() 建立五個並行 FreeRTOS Task，各自負責獨立的 I/O 通道：

Task 名稱	堆疊大小	功能說明
task_getMqttMessage	32 KB	MQTT 訊息收發、Broker 重連、Callback 處理
task_getRequest	16 KB	HTTP Web Chat 請求接收 (Port 81)

task_getRequestStream	16 KB	HTTP 視訊串流服務（Port 82）
task_time_scheduling	6 KB	每 60 秒掃描 schedule.json，觸發到期排程任務
task_theft_detection（選用）	6 KB	週期性防盜偵測，預設以 /* */ 包圍停用

⚠️ 共用網路資源競爭

多個 Task 共用同一個 WiFiSSLClient 實例。若同時發起 HTTPS 請求，將造成封包衝突導致解析失敗。

設計原則：任何 Task 在呼叫 Gemini API 前，須確保其他 Task 的連線已關閉或閒置。

1.4 系統提示詞組建邏輯（setup() 階段）

setup() 讀取 SD 卡設定檔後，組建三個不同範圍的 Gemini 系統提示詞字串，供不同情境使用：

變數名稱	組成內容	使用情境
systemContent	soul.md	純自然語言對話，不需工具路由
systemContentNoTools	soul.md + device.md + devicesRule	需要裝置情境但無工具呼叫的推理
systemContentTools	soul.md + device.md + devicesRule + skill.md + toolsDefinition	完整工具路由（主要執行路徑）

❗ 重要：soul.md 或 device.md 缺失會觸發自動重啟

若 soul.md 或 device.md 任一為空，setup() 呼叫 NVIC_SystemReset() 重啟裝置並停止初始化。這是防止 AI 在無系統人格和無裝置定義的狀態下運行的安全保護。

第二章 訊息處理與工具路由

2.1 訊息接收路徑

fuClaw 支援兩種訊息來源，統一進入同一個處理邏輯鏈：

來源	協定	進入點函式	傳輸特性
MQTT Broker	MQTT over TCP（或 TLS）	callback()	非同步 Callback，QoS 0
Web Chat	HTTP GET/POST（Port 81）	task_getRequest()	同步輪詢，支援 SSE 串流回應

兩者的訊息最終都傳入 `geminiChatRequest()`，取得 Gemini 回應後再送入 `handleAgentResponse()` 處理。

2.2 `handleAgentResponse()`：回應分流器

這是整個代理人邏輯的核心函式，負責解析 Gemini 的輸出並決定下一步動作。分流規則如下：

Gemini 輸出特徵	判斷結果	執行動作
以 { 開頭、以 } 結尾	單一 JSON 物件	ArduinoJson 解析 → 取出 method 與 params → <code>executeTool()</code>
以 [開頭、以] 結尾	JSON 陣列（多步驟）	循序執行每個 <code>tool_call</code> ；遇到不完整項目立即中止（最長有效前綴）
完全等於 "NONE"（區分大小寫）	工作流程完成哨兵	不執行任何動作，結束此輪工作流程
其他純文字	自然語言回覆	清理 Markdown 格式 → <code>replyUserMessage()</code> 傳送
JSON 解析失敗	格式錯誤	<code>Serial.println</code> 記錄，不觸發任何動作

前置字元清理（必要步驟）

`handleAgentResponse()` 在進行 JSON 解析前，執行以下字元替換，以處理 Gemini 偶發的輸出格式汙染：

```
message.replace("\\\\\"", "\\"); // 還原跳脫引號
message.replace("\\\\\"", "\\"); // 還原跳脫反斜線
message.replace("\\n", ""); // 移除 JSON 換行
message.replace("\\t", ""); // 移除 Tab
message.replace("\\-", "-"); // 還原 Markdown 跳脫
message.replace("\\*", "*"); // 還原 Markdown 跳脫
message.replace("\\_", "_"); // 還原 Markdown 跳脫
```

2.3 executeTool()：工具執行器

`executeTool()` 接收已通過 JSON 解析的 `method` 和 `params`，執行對應的硬體或系統動作。每次工具執行後，結果會寫入 `historicalMessages`（對話記憶）和 `executeToolHistory`（執行日誌）。

reCheck 旗標與工作流程評估

`executeTool()` 的第四個參數 `reCheck` 控制是否在工具執行後觸發 `evaluateWorkflowContinuation()`：

情境	reCheck 值	行為
JSON 陣列中的中間工具	false	執行後直接繼續，不詢問 Gemini
JSON 陣列中的最後一個工具	true	執行後呼叫 <code>evaluateWorkflowContinuation()</code>
單一 JSON 物件	true（預設）	執行後呼叫 <code>evaluateWorkflowContinuation()</code>

2.4 evaluateWorkflowContinuation()：工作流程自評

每個工具執行完畢後（當 `reCheck = true`），系統向 Gemini 發送一個評估提示詞，詢問工作流程是否完成。Gemini 的可能回應如下：

- 回傳 `NONE`：任務完成，流程結束。
- 回傳新的 `tool_call` JSON：需要繼續執行，再次進入 `handleAgentResponse()`。
- 回傳自然語言：任務完成，傳送最終說明給使用者。

✦ NONE 哨兵的格式要求

`NONE` 必須完全大寫且不含任何前後空白或標點。系統使用精確字串比對（`==`），Gemini 回傳 `"None"`、`"none"`、`"完成"` 等變體均不會被識別為哨兵，而是作為自然語言傳送。

在 `soul.md` 和 `skill.md` 中應明確強調此格式要求。

2.5 Gemini API 請求類型

類型	函式名稱	特性	使用情境
Chat	<code>geminiChatRequest()</code>	使用 <code>/v1beta/models/{model}:generateContent</code> ，帶完整對話歷史	主要推理路徑
Vision	<code>geminiVisionRequest()</code>	圖片以 Base64 <code>inline_data</code> 嵌入請求 JSON	<code>/still</code> 、 <code>/vision</code> 工具
Search	<code>geminiSearchRequest()</code>	啟用 <code>google_search</code> tool，取得接地搜尋結果	<code>/search</code> 工具

STT (語音)	sendAudioFileToGeminiSTT()	音訊以 Base64 inline_data 傳送，取得轉錄文字	Telegram 語音訊息
-------------	----------------------------	----------------------------------	------------------

Vision 請求的效能開銷

一張 320×240 JPEG（約 20 KB）Base64 編碼後約 27 KB，完整請求大小可達 50–100 KB。

Base64 編碼約需 27,000 次迭代，加上 HTTPS 握手與 API 回應，單次 /vision 呼叫總耗時通常為 5–15 秒。

語音檔案（Telegram OGG/Opus 格式）下載使用 HTTP/1.0，避免分塊傳輸編碼（Chunked Transfer Encoding）增加解碼複雜度。

第三章 持久化設定檔規格（SD 卡）

所有設定檔儲存於 SD 卡根目錄。系統在 `setup()` 階段依序讀取，並在對話過程中即時更新 `memory.md` 與 `schedule.json`。

3.1 env.json — 環境憑證

JSON 格式，包含所有外部服務的連線參數。系統透過 `setEnvironmentSettings()` 解析並套用。

```
{
  "device_name": "DEVICE_NAME",
  "wifi_ssid": "YOUR_SSID",
  "wifi_pass": "YOUR_PASSWORD",
  "mqtt_server": "mqttgo.io",
  "mqtt_port": "1883",
  "mqtt_user": "",
  "mqtt_password": "",
  "mqtt_subscribeTextTopic": "fuclaw/subscribe",
  "mqtt_publishTextTopic": "fuclaw/publish",
  "mqtt_publishImageTopic": "fuclaw/publishimage",
  "gemini_apikey": "AIzaSy...",
  "gemini_model": "gemini-2.5-flash-preview-05-20",
  "schedule_timeout": 10,
  "timezone": "Asia/Taipei"
}
```

欄位	類型	說明
device_name	String	裝置名稱
wifi_ssid / wifi_pass	String	WiFi 基礎設施憑證
mqtt_server / mqtt_port	String	MQTT Broker 位址與端口 (1883 = 明文 , 8883 = TLS)
mqtt_subscribeTextTopic	String	裝置訂閱的指令 Topic
mqtt_publishTextTopic	String	裝置發布文字回覆的 Topic
mqtt_publishImageTopic	String	裝置發布 JPEG 圖片的 Topic
gemini_apikey	String	Google Gemini API 金鑰
gemini_model	String	使用的 Gemini 模型識別字串
schedule_timeout	Integer	排程任務執行的容忍延遲 (分鐘)
timezone	String	IANA 時區字串，影響 RTC 對時與排程判斷

× 安全注意事項

`env.json` 含有 API Key 與 WiFi 密碼等敏感憑證，SD 卡遺失即等同憑證洩露。建議課後或測試後立即撤銷 Gemini API Key，並為測試環境使用專屬的 MQTT Topic 名稱。

3.2 soul.md — 代理人人格定義

純文字格式，內容直接覆蓋 `geminiRole` 變數，成為 Gemini 系統提示詞的第一層。此檔案決定 AI 的回答語氣、專業領域、決策風格，以及是否在特定情境下主動要求確認。

`soul.md` 為空或無法讀取時，`setup()` 將呼叫 `NVIC_SystemReset()` 重啟裝置。

3.3 device.md — 硬體裝置映射

定義 AI 被允許直接控制的 GPIO 腳位清單，是硬體安全的第一道防線。`devicesRule` 規定 AI 只能操作此清單中明確列出的腳位；對於未列出的裝置，AI 必須停止並向使用者請求澄清。

標準格式範例（HUB 8735 Ultra 完整配置）：

```
=====
CONFIRMED HARDWARE DEVICES
=====

HUB 8735 Ultra
- Green indicator LED: pin 25
- Blue indicator LED: pin 26
- Fill light LED: pin 13
  - analog output range: 0-255
- Function button: pin 12
  - digital input only, active-low

External Modules
- Window actuator (SG90 servo): pin 12
  - servo angle control, range: 0-180
- DHT11 Temperature & Humidity Sensor: PIN 20
  - Measures: temperature (°C) and humidity (%)
  - Read mode: single trigger, returns two integers
  - Temperature range: 0-50°C, Humidity: 20-90%RH
  - Physical Rule: sensor requires ~1s between reads
```

3.4 skill.md — 自主技能定義

定義 AI 可主動執行的複合工作流程（Skill）。每個 Skill 包含目標描述、機器可讀的 JSON 步驟序列，以及條件判斷邏輯。Skill 由 `periodicCheck Task` 或排程觸發，執行時豁免使用者確認要求。

Skill 定義必須嚴格使用 JSON 格式輸出步驟，不得包含自然語言說明。以防盜偵測 Skill 為例：

```
SKILL: anti_theft_detection
Goal: Detect human presence and trigger alert workflow.

Step 1: Analyze image for human presence
{
  "type": "tool_call",
  "method": "/vision",
  "params": {
    "query": "Is a person visible in the image?",
    "frames": true,
```



```

    "task": "If person detected, continue. If not, return NONE."
  }
}

Step 2: If person detected → alert sequence
[
  { "type": "tool_call", "method": "/still",
    "params": { "frames": false, "task": "NONE" } },
  { "type": "tool_call", "method": "/digitalwrite",
    "params": { "pin": 26, "pinmode": "digitalwrite", "value": 1 } },
  { "type": "tool_call", "method": "/delay",
    "params": { "milliseconds": 500 } },
  { "type": "tool_call", "method": "/digitalwrite",
    "params": { "pin": 26, "pinmode": "digitalwrite", "value": 0 } },
  { "type": "tool_call", "method": "/delay",
    "params": { "milliseconds": 500 } }
]

```

3.5 memory.md — 對話歷史持久化

儲存 Gemini API 所需的 messages 陣列格式，每次工具執行後自動附加並寫回 SD 卡。系統重啟後讀取此檔案，還原 historicalMessages 變數，實現跨重啟的記憶連續性。

⚠️ Heap 碎片化風險

隨對話增加，memory.md 持續成長，String 型態的頻繁 malloc/free 操作會導致 Heap 碎片化。

監控方式：發送 /getMemory 查看 xPortGetFreeHeapSize()（當前可用）與 xPortGetMinimumEverFreeHeapSize()（歷史最低）。

建議在 historicalMessages 長度超過 50,000 字元時，定期執行 /reset 或實作自動截斷機制。

3.6 schedule.json — 排程任務

由 /schedule 工具寫入，儲存使用者設定的定時任務列表。task_time_scheduling Task 每 60 秒掃描此檔案，比對當前 RTC 時間，觸發到期任務。排程任務執行時，自動豁免使用者確認要求（代表排程本身即為事先授權）。

第四章 工具 API 完整規格

所有工具均以 JSON `tool_call` 格式呼叫。Gemini 輸出的 JSON 由 `handleAgentResponse()` 解析後，送入 `executeTool()` 執行。以下為每個工具的完整請求/回應格式。

4.1 GPIO 輸出輸入

`/digitalwrite` — 數位輸出

```
// 請求
{ "type": "tool_call", "method": "/digitalwrite",
  "params": { "pin": <int>, "pinmode": "digitalwrite", "value": 0|1 } }

// 成功回應
{ "status": "success", "method": "digitalwrite", "workId": "<system>" }

// 錯誤回應 (value 不為 0 或 1)
{ "status": "error", "method": "digitalwrite",
  "reason": "invalid_digital_value", "workId": "<system>" }
```

`/analogwrite` — 類比輸出 (PWM)

```
// 請求
{ "type": "tool_call", "method": "/analogwrite",
  "params": { "pin": <int>, "pinmode": "analogwrite", "value": 0-255 } }

// 成功回應
{ "status": "success", "method": "analogwrite", "workId": "<system>" }
// value 由 constrain() 強制限制在 0-255，不會拋出範圍錯誤
```

`/digitalread` — 數位輸入

```
{ "type": "tool_call", "method": "/digitalread",
  "params": { "pin": <int>, "pinmode": "digitalread" } }

// 成功回應
{ "status": "success", "method": "digitalread", "value": 0|1, "workId":
"<system>" }
```

`/analogread` — 類比輸入 (ADC)

```
{ "type": "tool_call", "method": "/analogread",
  "params": { "pin": <int>, "pinmode": "analogread" } }

// 成功回應 (ADC 原始值)
{ "status": "success", "method": "analogread", "value": 0-1023, "workId":
"<system>" }
```

4.2 視覺與搜尋

/still — 拍照並傳送

```
{ "type": "tool_call", "method": "/still",
  "params": {
    "frames": true,    // true = 擷取新畫格；false = 使用上次快取
    "task": "NONE"    // 後續任務描述，無則填 NONE
  }
}
```

/vision — 拍照並由 Gemini 分析

```
{ "type": "tool_call", "method": "/vision",
  "params": {
    "query": "分析圖片中的內容",
    "frames": true,
    "task": "分析完成後的後續動作，無則填 NONE"
  }
}
```

`frames: false` 的設計目的：當 `/vision` 已擷取並快取圖片後，後續的 `/still` 使用 `frames: false` 可傳送同一張圖片，確保警示照片與偵測時間點一致，而非重新擷取（人可能已離開）。

/search — 網路接地搜尋

```
{ "type": "tool_call", "method": "/search",
  "params": {
    "query": "搜尋內容",
    "task": "搜尋結果取得後的後續動作，無則填 NONE"
  }
}
```

4.3 擴充感測器

/dht11 — DHT11 溫濕度感測器

```
// 請求
{ "type": "tool_call", "method": "/dht11",
  "params": { "pin": <int> } }
// AMB82-mini → pin 8；HUB 8735 Ultra → pin 20

// 成功回應
{ "status": "success", "method": "/dht11",
  "temperature": <float>, "humidity": <float>, "workId": "<system>" }

// 錯誤回應 (isnan() 檢測失敗)
{ "status": "error", "method": "/dht11",
```

```
"reason": "dht11_read_failed", "workId": "<system>" }
```

/servo — SG90 伺服馬達控制

```
// 請求
{ "type": "tool_call", "method": "/servo",
  "params": { "pin": 12, "angle": 0-180 } }

// 成功回應
{ "status": "success", "method": "/servo", "workId": "<system>" }

// 錯誤回應（角度超出範圍 或 未定義的腳位）
{ "status": "error", "method": "/servo",
  "reason": "invalid_servo_angle | undefined_servo_pin: <pin>",
  "workId": "<system>" }
```

4.4 系統管理工具

工具	說明	回應
/delay	暫停執行 0–10000 ms（constrain() 強制限制）	success + workId
/getMemory	回傳 Heap 可用空間與 historicalMessages 長度	success + 記憶體數值
/getLog	回傳 executeToolHistory 字串（Serial 輸出）	success
/reset	清除 historicalMessages，還原初始提示詞狀態	success，對話記憶清空
/chat	純自然語言回覆，包含 reply 文字	傳送給使用者
/reboot	呼叫 NVIC_SystemReset() 重啟裝置	裝置重啟
/schedule	新增排程任務至 schedule.json	success + workId
/getSchedule	取得所有排程任務	success + 任務列表
/modifySchedule	修改或刪除指定排程任務	success + workId
/clearSchedule	清除所有排程任務	success
/syncrtc	透過網路時間對時 RTC	success + 時間字串
/getrtc	取得 RTC 當前時間	success + 時間字串
/tcpSendMessage	透過 TCP 傳送訊息給另一個裝置或代理（Agent）	success + workId
/mqttSendMessage	透過 TCP 或 MQTT 傳送訊息給另一個裝置、代理（Agent）或所有 MQTT 訂閱者	
/mqttSendImage	透過 MQTT 傳送影像快照（Video Snapshot）給另一台 fuClaw 裝置	

	或所有 MQTT 訂閱者	
/telegramSendMessage	傳送訊息到 Telegram Bot	
/telegramSendImage	傳送影像到 Telegram Bot	
/lineSendMessage	傳送訊息到 LINE Bot	

第五章 新感測器擴充 SOP

fuClaw 採用「四步驟擴充模型」整合新硬體，以下以 DHT11 和 SG90 為完整示範，可直接作為其他感測器的擴充模板。

5.1 擴充四步驟

步驟	操作	修改位置
步驟 1	在 device.md 加入硬體描述與物理規則	SD 卡（不需重新燒錄）
步驟 2	在 toolsDefinition 字串中加入工具 JSON 規格	韌體原始碼（需重新燒錄）
步驟 3	撰寫底層 C++ 驅動函式 tool_xxx()	韌體原始碼（需重新燒錄）
步驟 4	在 executeTool() 中加入 else if 分支	韌體原始碼（需重新燒錄）

5.2 步驟一：device.md 擴充

在 device.md 的「External Modules」段落加入感測器描述。物理規則（範圍、讀取模式、時序要求）必須明確標注，Gemini 才能正確決策何時可以呼叫工具：

```
- DHT11 Temperature & Humidity Sensor
- Pin mapping:
  AMB82-mini: PIN 8
  HUB 8735 Ultra: PIN 20
- Measures: temperature (°C) and relative humidity (%)
- Read mode: single trigger, returns two integer values
- Temperature range: 0-50°C
- Humidity range: 20-90% RH
- Physical Rule: Values are integers.
  Sensor requires ~1s between reads.
```

5.3 步驟二：toolsDefinition 工具規格

在韌體的 toolsDefinition 字串中加入新工具的 JSON 請求/回應格式。此定義送給 Gemini 作為提示詞，告知 AI 何時及如何呼叫此工具：

```
Reading the DHT11 temperature and humidity sensor:
{
  "type": "tool_call",
  "method": "/dht11",
  "params": {
    "pin": "<Device pin number."
      "If the user does not specify a pin, ask first.>"
  }
}

Success response:
```

```
{ "status": "success", "method": "/dht11",
  "temperature": <value>, "humidity": <value>,
  "workId": "<system-provided>" }

Error response:
{ "status": "error", "method": "/dht11",
  "reason": "<error reason>", "workId": "<system-provided>" }
```

5.4 步驟三：C++ 驅動函式

撰寫底層驅動，處理硬體讀取與 JSON 回應組建。以 DHT11 為例：

```
#include "DHT.h"
#define DHTTYPE DHT11
DHT dht(DHTPIN, DHTTYPE);    // DHTPIN 依開發板設定

String tool_dht11(int pin, String workId) {
  float h = dht.readHumidity();
  float t = dht.readTemperature();
  if (isnan(h) || isnan(t)) {
    return "{\"status\":\"error\","
      "\"method\":\"/dht11\","
      "\"reason\":\"dht11_read_failed\","
      "\"workId\":\"" + workId + "\"}";
  }
  return "{\"status\":\"success\","
    "\"method\":\"/dht11\","
    "\"temperature\":\"" + String(t) + "\",\"
    \"humidity\":\"" + String(h) + "\",\"
    \"workId\":\"" + workId + "\"}";
}

// setup() 中加入初始化：
delay(1000);    // 等待感測器穩定
dht.begin();
```

SG90 伺服馬達驅動函式以相同模式撰寫，額外需要驗證角度範圍（0–180）並在超出範圍時回傳 `invalid_servo_angle` 錯誤：

```
#include <AmebaServo.h>
AmebaServo servo12;

String tool_servo(AmebaServo &servo, int pin, int angle, String workId) {
  if (!servo.attached()) servo.attach(pin);
  if (angle < 0 || angle > 180) {
    return "{\"status\":\"error\",\"method\":\"/servo\","
      "\"reason\":\"invalid_servo_angle\","
      "\"workId\":\"" + workId + "\"}";
  }
  servo.write(angle);
  return "{\"status\":\"success\",\"method\":\"/servo\","
    "\"workId\":\"" + workId + "\"}";
}

// setup() 中：
servo12.attach(12);
```

5.5 步驟四：executeTool() 分支

在 executeTool() 函式中，於現有的 else if 鏈中加入新工具的分支。以下為 DHT11 和 SG90 的完整分支範例：

```
else if (command == "/dht11") {
    int pin = params["pin"].as<int>();
    String response = tool_dht11(pin, workId);

    historicalMessages += buildGeminiMessage("user", command + timestamps);
    historicalMessages += buildGeminiMessage("model", response + timestamps);
    executeToolHistory += workId + " " + command +
        " [ " + String(pin) + " | " + response + " ]\n";
    evaluateWorkflowContinuation(workId, reCheck);
}
else if (command == "/servo") {
    int pin = params["pin"].as<int>();
    int angle = params["angle"].as<int>();
    String response = "";
    if (pin == 12)
        response = tool_servo(servo12, pin, angle, workId);
    else
        response = "{\"status\":\"error\","
            "\"reason\":\"undefined_servo_pin: " + String(pin) + "\",\"
            \"workId\":\"" + workId + "\"}";

    historicalMessages += buildGeminiMessage("user", command + timestamps);
    historicalMessages += buildGeminiMessage("model", response + timestamps);
    executeToolHistory += workId + " " + command +
        " [ " + String(pin) + " | " + String(angle) + " ]\n";
    evaluateWorkflowContinuation(workId, reCheck);
}
```


第六章 硬體安全規範

6.1 裝置控制政策（Global Device Control Policy）

所有直接由使用者請求觸發的硬體輸出動作，執行前必須獲得明確確認。以下為三層優先順序：

優先級	規則	說明
1（最高）	安全限制	未確認的腳位絕不操作，未定義的裝置絕不猜測
2	排程任務執行規則	排程任務代表事先授權，執行時免除確認要求
3	自主工作流程規則	Skill 觸發的動作（如防盜偵測）免除確認要求
4（最低）	一般確認要求	使用者直接請求的硬體控制須先確認

6.2 原子執行規則（Atomic Execution Rule）

每次工具呼叫必須是最小不可分割的單元：

- 一個腳位（One Pin）：不能在同一個 `tool_call` 中同時操作多個 GPIO。
- 一個操作（One Operation）：不能同時讀取和寫入。
- 一個數值（One Value）：`digitalwrite` 只能是整數 0 或 1，不能是字串或複合指令。

多步驟操作必須使用 JSON 陣列，由系統循序執行。

6.3 數值驗證規則

工具	有效範圍	越界行為
<code>/digitalwrite</code>	0 或 1（整數）	回傳 <code>invalid_digital_value</code> ，不執行
<code>/analogwrite</code>	0–255（整數）	<code>constrain()</code> 強制限制，不報錯
<code>/delay</code>	0–10000（毫秒）	<code>constrain()</code> 強制限制，不報錯
<code>/servo</code>	0–180（角度整數）	回傳 <code>invalid_servo_angle</code> ，不執行
<code>/digitalread</code>	無需輸入數值	純讀取，回傳 0 或 1
<code>/analogread</code>	無需輸入數值	純讀取，回傳 ADC 原始值 0–1023

6.4 輸出裝置狀態查詢規則

當使用者詢問輸出裝置（LED、繼電器等）的當前狀態時，AI 必須從對話歷史和工具執行歷史判斷，不得呼叫 `/digitalread` 或 `/analogread` 來讀取輸出腳位的電氣狀態。

i 為何不用 `digitalread` 查輸出狀態？

裝置邏輯狀態（on/off）與 GPIO 電氣電位（0/1）是不同概念。直接讀取輸出腳位的電位可能因硬體差異或外部干擾而產生誤判，且無法反映 PWM 亮度等類比狀態。從對話歷史判斷是更可靠的設計。

第七章 部署與維運

7.1 首次部署步驟

1. 格式化 SD 卡（FAT32），建立五個設定檔：env.json、soul.md、device.md、skill.md、memory.md。
2. 填寫 env.json：WiFi 憑證、MQTT Broker 參數、Gemini API Key 與模型名稱。
3. 依實際硬體接線更新 device.md。
4. 在 Arduino IDE 安裝 Realtek AmebaPro2 SDK 和必要函式庫（DHT、AmebaServo、ArduinoJson、PubSubClient）。
5. 燒錄韌體至開發板，插入 SD 卡，接通電源。
6. 觀察 Serial Monitor（115200 baud）確認啟動序列：env.json 讀取 → WiFi 連線 → 各設定檔載入 → FreeRTOS Task 建立。
7. LED 閃爍三次（WiFi 連線成功），MQTT 訂閱完成後系統進入就緒狀態。

7.2 Web 管理介面

系統同時啟動兩個 HTTP 服務，提供 Web 介面管理：

URL	功能
http://<device_ip>:81	fuClaw 管理介面（設定查看、Web Chat、排程管理）
http://<device_ip>:82	即時視訊串流（MJPEG）
http://192.168.1.1:81	AP 模式下的固定位址（WiFi 未連線時）

7.3 Serial Monitor 啟動序列解讀

```
env.json len: 312           // env.json 讀取成功
Soul.md len: 128            // soul.md 讀取成功
device.md len: 847          // device.md 讀取成功
skill.md len: 623           // skill.md 讀取成功
memory.md len: 2048         // 對話歷史還原
AP ssid : fuclaw            // AP 模式 SSID
AP password : 12345678      // AP 模式密碼
fuClaw Manager: http://192.168.x.x:81
Video stream: http://192.168.x.x:82
RTC START: 2026-06-07 ..    // RTC 對時完成
```

⚠ 啟動失敗常見原因

soul.md 或 device.md 長度為 0：系統會輸出 "System configuration failed" 並在 5 秒後重啟。請確認 SD 卡根目錄有這兩個檔案且內容不為空。

WiFi 連線失敗：裝置仍可運行（AP 模式），但 Gemini API 和 MQTT 無法使用。

MQTT 連線失敗：task_getMqttMessage 會持續嘗試重連，不影響 Web Chat 功能。

7.4 記憶體監控與維護

指令	功能	建議閾值
/getMemory	查看 Heap 剩餘空間與歷史最低值	低於 50 KB 時執行 /reset
/reset	清除對話歷史，釋放 historicalMessages 佔用的 Heap	historicalMessages > 50,000 字元時
/getLog	查看工具執行歷史記錄（Serial 輸出）	除錯用途
/reboot	重啟裝置，完全釋放 Heap 碎片	Heap 嚴重碎片化時

7.5 已知技術限制

限制項目	說明	緩解策略
Heap 碎片化	String 型態的頻繁 malloc/free，長時間運行後可能導致記憶體不足	定期 /reset；監控 xPortGetMinimumEverFreeHeapSize()
網路資源競爭	多個 FreeRTOS Task 共用同一 WiFiSSLClient	架構設計確保同時只有一個 Task 發起 HTTPS 請求
Gemini 輸出不確定性	Temperature=1.0 時，偶發 JSON 格式汙染或 NONE 哨兵變體	handleAgentResponse() 前置清理；在 soul.md 強調格式要求
Vision 任務高延遲	Base64 編碼 + HTTPS + API 回應，通常 5–15 秒	非即時場景使用；不適合需要毫秒級反應的安全系統
ArduinoJson 緩衝限制	DynamicJsonDocument 設為 8 KB；超大 JSON 可能解析失敗	調整 geminiMaxOutputTokens；分段執行複雜任務

附錄 快速參考

A. GPIO 腳位對照表

開發板	裝置	腳位	控制模式	有效範圍
AMB82-mini	綠色 LED	24	digitalwrite	0 / 1
AMB82-mini	藍色 LED	23	digitalwrite	0 / 1
AMB82-mini	DHT11	8	dht11（專屬工具）	—
HUB 8735 Ultra	綠色 LED	25	digitalwrite	0 / 1
HUB 8735 Ultra	藍色 LED	26	digitalwrite	0 / 1
HUB 8735 Ultra	補光 LED	13	analogwrite	0–255
HUB 8735 Ultra	功能按鈕	12	digitalread（active-low）	0=按下
HUB 8735 Ultra	DHT11	20	dht11（專屬工具）	—
外部模組	緊急按鈕	1	digitalread（active-high）	1=按下
外部模組	光線感測器	2	analogread	0–1023
外部模組	警示燈	11	analogwrite	0–255
外部模組	SG90 伺服馬達	12	servo（專屬工具）	0–180°

B. 擴充感測器檢查清單

- `device.md`：加入感測器描述、腳位映射、物理規則（範圍、時序）
- `toolsDefinition`：加入工具 JSON 請求格式與成功/錯誤回應格式
- 韌體：撰寫 `tool_xxx()` 驅動函式，包含錯誤處理
- 韌體：在 `executeTool()` 加入 `else if` 分支
- 韌體：在 `setup()` 加入初始化程式碼
- 燒錄並測試：確認 Serial Monitor 無錯誤，工具回應格式正確

C. 常見錯誤代碼

錯誤原因	錯誤碼	處置方式
digitalwrite 數值不是 0 或 1	invalid_digital_value	確認 Gemini 輸出的 value 欄位
servo 角度超出 0–180	invalid_servo_angle	在 <code>soul.md</code> 強調角度範圍限制
servo 腳位未定義	undefined_servo_pin: <pin>	在 <code>device.md</code> 加入對應腳位定義
DHT11 讀值失敗	dht11_read_failed	確認接線；兩次讀取間隔 ≥1 秒

JSON 解析失敗	(Serial 輸出) JSON parse failed	降低 geminiTemperature ；回覆 Continue 重 試
記憶體不足	malloc failed	立即執行 /reset ；考慮 /reboot
Gemini API 逾時	Connection failed	確認 WiFi 品質 ；確認 API Key 配額

fuClaw AIoT 技術參考手冊 作者：傅仲儀 (ChungYi Fu) | 高雄，台灣 | <https://github.com/fustyles/fuClaw>